

Hadoop MapReduce 原理详解

第七讲：MapReduce 编程模型

Driver - Mapper - Reducer - Writable - Context - 完整 Java WordCount

MapReduce 编程模型：你只定义“怎么算”，框架负责“怎么分布式地算”



图 1 MapReduce 编程模型：业务逻辑与分布式运行机制的分工

本讲核心问题：前六讲我们一直在问“Hadoop 内部发生了什么”。这一讲换一个视角：作为程序员，我们到底要写哪些代码？这些看起来很短的 Java 代码，分别对应前六讲中的哪些底层机制？

一、本讲学习目标

- 理解 Driver、Mapper、Reducer 三个部分分别承担什么责任。
- 看懂 Mapper<KEYIN, VALUEIN, KEYOUT, VALUEOUT> 四个泛型参数，而不是机械照抄模板。
- 理解 LongWritable、Text、IntWritable 等 Writable 类型为什么存在。
- 理解 Context 为什么是 Mapper/Reducer 与 Hadoop 运行时之间的关键接口。
- 能够把完整 Java WordCount 的每一行代码，对应到 InputSplit、RecordReader、Shuffle、Sort、Reduce 等底层机制。
- 能够在 Hadoop 3.3.5 + JDK 1.8 环境中编译、打包、提交并观察一个 MapReduce Job。
- 学会根据“我要按什么 Key 分组”设计一个简单 MapReduce 程序。

先建立一个总心智模型：MapReduce 编程并不是让你自己写“分布式通信代码”。你只定义：一条输入记录怎样变成中间键值对，以及同一个 Key 的一组 Value 怎样聚合。数据切分、任务调度、网络传输、排序和失败重试由 Hadoop 负责。

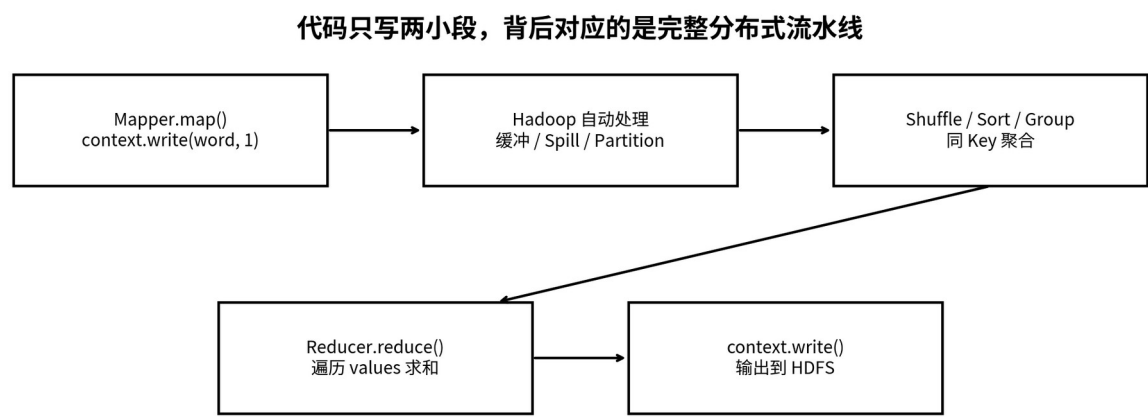


图 2 两段业务代码背后对应的是完整的分布式流水线

二、一个 MapReduce 程序到底由什么组成

经典 Hadoop MapReduce 程序通常可以先分成三个角色：Driver、Mapper、Reducer。把它们理解成“总控配置 - 局部处理 - 分组聚合”最容易。

角色	你写什么	它不负责什么	对应前面原理
Driver	描述整个 Job：使用哪个 Mapper/Reducer、输入输出路径、类型等	不逐条读取业务数据	Job 提交、YARN 调度、InputSplit 计算
Mapper	处理一条输入记录，产生 0 个、1 个或多个中间键值对	不负责把同 Key 数据集中到一起	RecordReader -> map() -> Map 输出缓冲
Reducer	处理一个 Key 以及属于该 Key 的一组 Values	不负责自己去遍历所有 Mapper 找数据	Shuffle/Sort/Group 后的 reduce()

最容易犯的错误：把 Mapper 理解成“第一半算法”、Reducer 理解成“第二半算法”并不准确。更准确的理解是：Mapper 负责局部转换；Hadoop 按 Key 重新组织数据；Reducer 负责按 Key 聚合。中间那段 Shuffle 才是 MapReduce 的关键分布式机制。

三、Mapper 的四个泛型参数到底是什么意思

```
public static class TokenizerMapper
    extends Mapper<LongWritable, Text, Text, IntWritable> {
    ...
}
```

很多初学者看到四个类型会直接背模板。更好的办法是把它读成一句话：这个 Mapper 接收什么类型的输入 Key 和 Value，又输出什么类型的 Key 和 Value。

位置	WordCount 中的类型	真实含义
KEYIN	LongWritable	一行文本在文件中的起始字节偏移量
VALUEIN	Text	RecordReader 读出来的一行文本
KEYOUT	Text	Mapper 输出的单词
VALUEOUT	IntWritable	Mapper 为每个单词输出的计数 1

WordCount 的类型链：四个泛型参数不是装饰，而是数据契约

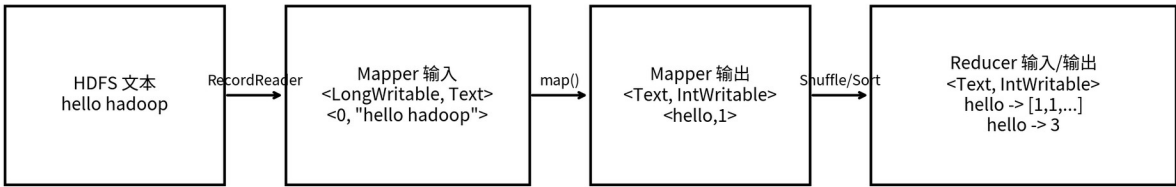


图 3 WordCount 的输入输出类型链

为什么 KEYIN 不是“第几行”：TextInputFormat 的默认 RecordReader 使用字节偏移量作为 Key。例如第一行从第 0 字节开始，下一行可能从第 18 字节开始。这个 Key 更多是记录定位信息，WordCount 本身通常不会使用它。

四、Reducer 的四个泛型参数

```
public static class IntSumReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {
    ...
}
```

Reducer 的输入类型必须与 Map 端经过 Shuffle 后的中间 Key/Value 类型兼容。WordCount 的 Mapper 输出 <Text, IntWritable>，因此 Reducer 看到的就是某个 Text Key 加上一组 IntWritable Values。

Reducer 泛型	WordCount 类型	理解方式
KEYIN	Text	例如 hello
VALUEIN	IntWritable	hello 对应的一串 1
KEYOUT	Text	最终仍输出 hello
VALUEOUT	IntWritable	最终次数，例如 37

五、为什么 Hadoop 不直接使用 String 和 Integer

MapReduce 的中间数据需要不断在内存、磁盘和网络之间移动。Java 对象本身不能直接“沿着网线传过去”，必须先序列化为字节。Hadoop 因此设计了 Writable 序列化体系。

Java 常见类型	Hadoop Writable	典型用途
long / Long	LongWritable	偏移量、长整数
int / Integer	IntWritable	计数、整数指标
String	Text	字符串、单词、ID
boolean	BooleanWritable	布尔值
无有效值	NullWritable	只需要 Key、不需要 Value 的场景

底层逻辑可以简化成：Java 对象 -> Writable.write() -> 字节流 -> 网络/磁盘 -> Writable.readFields() -> 恢复对象。Hadoop 的 Writable 设计目标之一就是让这种高频率序列化更紧凑、更可控。

一个重要细节：Writable 对象经常被框架复用。初学阶段先记住：不要想当然地把框架传入的 key/value 对象长期保存到集合里而不复制，因为后续迭代时同一个对象可能被重新填充。

六、Context 到底是什么

Mapper 和 Reducer 方法里都有一个 Context 参数。它不是普通“上下文”概念，而是业务代码与 Hadoop 运行时之间的接口。你既可以通过它向框架输出数据，也可以读取配置、更新 Counter、了解任务信息。

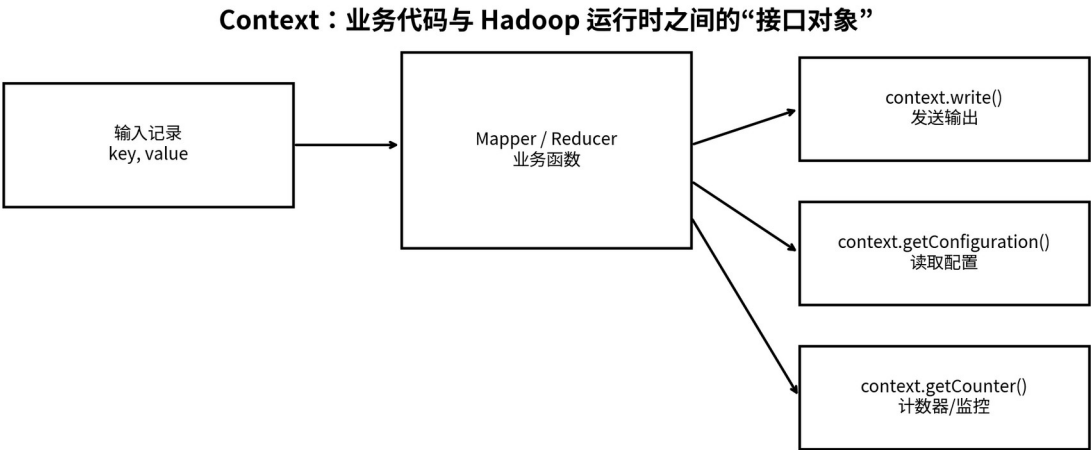


图 4 Context 连接了业务函数与 Hadoop 运行时

```
context.write(word, one); // 输出中间键值对
context.getConfiguration().get("my.param"); // 读取 Job 配置
context.getCounter("DATA", "BAD_LINE").increment(1); // 自定义计数器
```

关键理解：context.write() 并不是“立刻把数据写入 HDFS”。在 Mapper 中，它通常只是把中间结果交给 Map 输出收集器，后面还要经历序列化、缓冲、Spill、Partition、Sort、Shuffle。

七、逐行看懂 WordCount Mapper

```
public static class TokenizerMapper
    extends Mapper<LongWritable, Text, Text, IntWritable> {

    // 值为 1 的 IntWritable 对象可重复使用，减少反复创建对象的开销。
    private static final IntWritable ONE = new IntWritable(1);

    // 用于保存当前单词。这里同样复用 Text 对象。
    private final Text word = new Text();

    @Override
    protected void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {

        // value 是 RecordReader 交给我们的“一行文本”。
        String line = value.toString();

        // 为了教学清晰，用空白字符切词。真实文本处理会更复杂。
        String[] tokens = line.trim().split("\\s+");

        for (String token : tokens) {
            if (!token.isEmpty()) {
                word.set(token);

                // 把 <单词, 1> 交给 Hadoop 的 Map 输出收集器。
                // 这里不会直接调用 Reducer，也不会直接写 HDFS。
                context.write(word, ONE);
            }
        }
    }
}
```

7.1 一条输入记录实际上怎样走到 map()

假设 HDFS 文件中有一行："hello hadoop hello"。TextInputFormat 先决定 InputSplit，RecordReader 再把该行转换成类似 <0, "hello hadoop hello"> 的记录，然后框架调用一次 map(key, value, context)。

输入: <0, "hello hadoop hello">

map() 内部切词后:
hello -> context.write("hello", 1)
hadoop -> context.write("hadoop", 1)
hello -> context.write("hello", 1)

7.2 Mapper 可以输出几条记录

不是“一条输入必须对应一条输出”。Mapper 可以输出 0 条（过滤掉）、1 条（转换）、多条（例如一行拆成多个单词）。这也是 Map 阶段非常灵活的地方。

业务动作	输入 1 条记录	Mapper 可能输出
过滤无效日志	1 条	0 条
字段转换	1 条	1 条
WordCount 切词	1 行	多个 <word,1>
倒排索引	1 个文档	多个 <term,docId>

八、最重要的一段代码：你其实没有写 Shuffle 代码

观察 WordCount 程序会发现一个很反直觉的事实：Mapper 和 Reducer 中间没有任何“把 hello 发给 Reducer”的 Java 代码。因为这部分完全由 Hadoop 框架接管。

// 你只写：
context.write(new Text("hello"), new IntWritable(1));

// Hadoop 在背后自动完成：
序列化 -> 内存缓冲 -> Spill -> Partition -> Sort -> 网络 Shuffle
-> Reduce 端 Merge -> Group -> hello -> [1, 1, 1, ...]

这就是 MapReduce 编程模型的价值：如果没有框架，你需要自己解决：哪台机器保存了 hello、网络连接怎么建、机器挂了怎么重试、数据怎么排序、相同 Key 怎样聚到一起。MapReduce 把这些“分布式系统复杂性”封装在框架里。

九、逐行看懂 Reducer

```
public static class IntSumReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {

    private final IntWritable result = new IntWritable();

    @Override
    protected void reduce(Text key, Iterable<IntWritable> values,
                          Context context)
        throws IOException, InterruptedException {

        int sum = 0;

        // values 是当前 key 对应的一组值。
        // 例如 key="hello" 时，可能是 [1, 1, 1, 1]。
        for (IntWritable value : values) {
            sum += value.get();
        }

        result.set(sum);

        // Reducer 的输出最终由 OutputFormat 写入 HDFS。
        context.write(key, result);
    }
}
```

注意 `reduce()` 不是“每条 Map 输出调用一次”。它是按分组后的 Key 调用。例如 hello 有 100 万条中间记录，框架先把它们按 Key 聚到一起，然后对 hello 调用一次 `reduce()`，`values` 代表这个 Key 的所有值。

为什么参数是 `Iterable` 而不是 `List`： 因为一个热门 Key 可能有上百万甚至上亿个 Value。框架希望你以流式迭代方式读取，而不是强迫把所有 Value 先塞进一个巨大 List 再处理。

十、Driver：它描述整个 Job，而不是处理数据

Driver 的本质：描述一个 Job，而不是亲自处理每一条数据



图 5 Driver 的主要职责

```

public static void main(String[] args) throws Exception {

    // 1. 读取 Hadoop 配置，例如 core-site.xml、hdfs-site.xml、yarn-site.xml。
    Configuration conf = new Configuration();

    // 2. 创建一个 Job 描述对象。
    Job job = Job.getInstance(conf, "word count");

    // 3. 告诉 Hadoop：这个 Job 的主类在哪里，用于定位 Jar。
    job.setJarByClass(WordCount.class);

    // 4. 注册 Mapper 和 Reducer。
    job.setMapperClass(TokenizerMapper.class);
    job.setReducerClass(IntSumReducer.class);

    // 5. WordCount 可以安全使用 Reducer 逻辑做本地预聚合。
    job.setCombinerClass(IntSumReducer.class);

    // 6. 声明最终输出 Key / Value 类型。
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    // 7. 告诉 Job：HDFS 上从哪里读、写到哪里。
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    // 8. 提交 Job，并让客户端等待执行结束。
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}

```

10.1 setOutputKeyClass() 为什么不等于 Mapper 输出类型

setOutputKeyClass()/setOutputValueClass() 默认描述的是 Job 最终输出，也就是 Reducer 输出。如果 Mapper 输出类型与 Reducer 输出类型不同，还需要显式设置 setMapOutputKeyClass() 和 setMapOutputValueClass()。

```

job.setMapOutputKeyClass(Text.class);
job.setMapOutputValueClass(IntWritable.class);

job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);

```

类型不一致会怎样：很多 MapReduce 初学错误并不是算法错，而是“类型链断了”。例如 Mapper 实际输出 Text/IntWritable，却在 Job 中配置成 Text/Text，往往会在运行阶段出现类型转换或序列化错误。

十一、完整 Java WordCount：把全部代码放在一起

下面代码故意保留较多注释。学习时先看“数据类型与 context.write()”，再看 Driver 配置；不要一开始纠结 Java 语法细节。

```

import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
        extends Mapper<LongWritable, Text, Text, IntWritable> {

        private static final IntWritable ONE = new IntWritable(1);
        private final Text word = new Text();

        @Override
        protected void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {

            String line = value.toString();
            String[] tokens = line.trim().split("\\s+");

            for (String token : tokens) {
                if (!token.isEmpty()) {
                    word.set(token);
                    context.write(word, ONE);
                }
            }
        }
    }
}

```

到这里，程序只完成“局部转换”：每一行文本被拆成多个 <word,1>。接下来 Hadoop 会自动进行 Map 端缓冲、分区、排序和 Shuffle。

十一、完整 Java WordCount（续）

```

public static class IntSumReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {

    private final IntWritable result = new IntWritable();

    @Override
    protected void reduce(Text key, Iterable<IntWritable> values,
        Context context)
        throws IOException, InterruptedException {

        int sum = 0;
        for (IntWritable value : values) {
            sum += value.get();
        }

        result.set(sum);
        context.write(key, result);
    }
}

public static void main(String[] args) throws Exception {
    if (args.length != 2) {
        System.err.println("Usage: WordCount <input> <output>");
        System.exit(2);
    }

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");

    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}

```

为什么这里设置 Combiner：求和满足局部聚合再聚合仍然等价：1+1+1 可以先在 Mapper 本地变成 3，再发送给 Reducer。因此 WordCount 很适合 Combiner。但不要形成“Reducer 一定可以直接拿来当 Combiner”的错误习惯。平均值等问题需要额外设计。

十二、在 Hadoop 3.3.5 虚拟机中真正跑起来

下面以 Ubuntu 22.04、JDK 1.8、Hadoop 3.3.5 为基线。每次重新打开虚拟机后，不要假设 HDFS/YARN 已经运行，先启动并做健康检查。

12.1 启动 HDFS 与 YARN，并检查状态

```
# 启动 HDFS。
start-dfs.sh

# 启动 YARN。
start-yarn.sh

# 查看 Java 进程。单机伪分布式环境通常应看到：
# NameNode、DataNode、SecondaryNameNode、ResourceManager、NodeManager。
jps

# 检查 HDFS DataNode 状态。
hdfs dfsadmin -report

# 检查 YARN NodeManager 是否已注册。
yarn node -list
```

12.2 准备源码目录

```
mkdir -p ~/mr-wordcount/src ~/mr-wordcount/classes
cd ~/mr-wordcount

# 用你熟悉的编辑器创建源码。
# 文件名必须与 public class WordCount 一致。
vim src/WordCount.java
```

12.3 编译 Java 源码

```
cd ~/mr-wordcount

# hadoop classpath 会输出编译 Hadoop 程序所需的所有 Jar 路径。
javac -classpath "${hadoop classpath}" \
    -d classes \
    src/WordCount.java

# 查看是否生成 WordCount.class 及内部类 class 文件。
find classes -type f -name "*.class" -print
```

十二、在虚拟机中真正跑起来（续）

12.4 打包为 Jar

```
cd ~/mr-wordcount

# -C classes . 表示先切换到 classes 目录，再把其中内容加入 Jar。
jar -cvf wordcount.jar -C classes .

# 查看 Jar 内容。
jar -tf wordcount.jar
```

12.5 准备 HDFS 输入数据

```
cd ~/mr-wordcount

printf "hello hadoop hello\nhadoop mapreduce\nhello bigdata\n" > data.txt

# 创建 HDFS 输入目录。
hdfs dfs -mkdir -p /user/$USER/wc_input

# 上传测试文件；-f 表示目标同名文件存在时覆盖。
hdfs dfs -put -f data.txt /user/$USER/wc_input/

# 先确认 HDFS 中的内容确实存在。
hdfs dfs -cat /user/$USER/wc_input/data.txt
```

12.6 提交 Job

```
# Hadoop 要求输出目录在 Job 启动前不能已经存在。
# 为了重复实验，先删除旧输出。
hdfs dfs -rm -r -f /user/$USER/wc_output

# 提交 MapReduce Job。
hadoop jar ~/mr-wordcount/wordcount.jar \
    WordCount \
    /user/$USER/wc_input \
    /user/$USER/wc_output
```

12.7 查看结果

```
hdfs dfs -ls /user/$USER/wc_output

hdfs dfs -cat /user/$USER/wc_output/part-r-00000
```

对于上面的测试数据，结果应包含 bigdata=1、hadoop=2、hello=3、mapreduce=1。默认文本输出以 Tab 分隔 Key 和 Value。

十三、运行时应该观察什么，而不是只看“成功”

真正学会 MapReduce，不能只看到最终 part-r-00000。你应该把控制台日志与前六讲的机制一一对应。

你看到的现象	应该联想到的原理
map 0% -> 100%	MapTask 正在处理各 InputSplit
reduce 0% 开始变化	Reducer 已启动，Shuffle/Sort/Reduce 阶段在推进
Map output records	Mapper 一共输出多少条中间记录
Map output bytes	Shuffle 前中间数据量的重要指标
Combine input/output records	Combiner 是否减少了中间记录
Reduce input groups	有多少个不同 Key 分组进入 Reducer
Reduce input records	Reducer 实际接收多少条 Value 记录
HDFS bytes written	最终输出到 HDFS 的字节量

13.1 YARN Web UI 看什么

如果虚拟机网络配置允许，可在浏览器查看 ResourceManager Web UI（常见端口 8088）。进入 Application 后重点看 ApplicationMaster、Map/Reduce task attempts、Container、Counters 和日志。不要把 UI 只当“看进度条”的页面，它其实是理解 Job 生命周期最直观的工具之一。

13.2 为什么 Counters 特别值得学

Counters 是 MapReduce 中低成本的可观测性机制。例如你做日志清洗时，可以统计多少行正常、多少行缺字段、多少行解析失败。这样不用等到程序结束后再猜“数据是不是被过滤太多”。

```
public enum MyCounter { BAD_LINE, VALID_LINE }

// 遇到坏数据：
context.getCounter(MyCounter.BAD_LINE).increment(1);

// 正常数据：
context.getCounter(MyCounter.VALID_LINE).increment(1);
```

十四、初学者最常见的运行错误

现象	常见原因	怎么判断/处理
Output directory already exists	Hadoop 不允许直接覆盖 Job 输出目录	运行前 <code>hdfs dfs -rm -r -f 输出目录</code>
ClassNotFoundException	Jar 中没有目标类，或主类名写错	<code>jar -tf wordcount.jar</code> 检查类文件
找不到 Hadoop 类，javac 报错	编译 classpath 不完整	使用 <code>-classpath "\$(hadoop classpath)"</code>
类型转换/序列化异常	Mapper/Reducer/Job 配置类型链不一致	逐项核对四个泛型与 <code>set...Class</code>
Job 一直 ACCEPTED	YARN 没资源或 NodeManager 未注册	<code>yarn node -list</code> ，查看 RM UI
输入路径不存在	HDFS 路径与 Linux 本地路径混淆	<code>hdfs dfs -ls</code> 检查
结果大小写分开统计	WordCount 没做标准化	Mapper 中 <code>token.toLowerCase()</code>
标点与单词黏在一起	<code>split("\\s+")</code> 只按空白切分	增加正则清洗或 tokenizer

HDFS 路径和 Linux 路径一定要分清： `~/mr-wordcount/data.txt` 是虚拟机本地文件；`/user/$USER/wc_input` 是 HDFS 命名空间中的路径。hadoop jar 的 `FileInputFormat` 默认读取 HDFS（取决于 `fs.defaultFS` 配置），不能把两种路径混为一谈。

十五、从 WordCount 迁移到真实问题：先问“Key 应该是谁？”

设计 MapReduce 程序时，比“代码怎么写”更关键的是业务建模。先确定你最终希望按谁分组，那个字段往往就是 Mapper 输出 Key。

现实问题	Mapper 输出 Key	Mapper 输出 Value	Reducer 做什么
统计每个 URL 的访问次数	URL	1	求和
统计每个城市销售额	city	amount	求和
求每个班级最高分	classId	score	取 max
建立单词倒排索引	word	docId	收集文档 ID
按用户汇总消费	userId	amount	求和/统计

一个非常实用的设计口诀：先问“最后我要按什么东西得到一组数据？” -> 把它作为 Key；再问“每条原始记录为这个 Key 贡献什么信息？” -> 把它作为 Value；最后问“这一组 Value 要怎样聚合？” -> 这就是 Reducer。

十六、三个迁移练习

练习 1：统计 HTTP 状态码次数

假设日志每行包含 statusCode，例如 200、404、500。请先不写代码，回答：Mapper 输出 Key 是什么？Value 是什么？Reducer 做什么？

示例思路：

200 -> 1

404 -> 1

200 -> 1

Shuffle 后：

200 -> [1, 1, ...]

404 -> [1, ...]

Reducer：分别求和

练习 2：统计每个学生的总分

输入格式：studentId, course, score。目标是每个学生的总分。这里的 Key 应该是 studentId，Value 是 score，Reducer 做求和。注意：这和 WordCount 的结构完全一样，只是“单词”换成“学生 ID”，“1”换成“成绩”。

练习 3：求每个城市的最高温度

输入格式：date, city, temperature。Mapper 输出 <city, temperature>，Reducer 对每个城市遍历所有温度，维护 max，而不是 sum。你会发现，MapReduce 的骨架几乎没变，真正改变的是业务聚合函数。

十七、本讲最终心智模型

MapReduce 编程模型：你只定义“怎么算”，框架负责“怎么分布式地算”



图 6 重新回到本讲最核心的分工

1. Driver：声明“这个 Job 长什么样”，然后提交给 Hadoop。
2. Mapper：把一条输入记录转换成 0 个或多个中间 <Key, Value>。
3. Hadoop：自动完成缓冲、Spill、Partition、Shuffle、Sort、Grouping。
4. Reducer：对同一个 Key 的一组 Values 做聚合。
5. Writable：让这些 Key/Value 能高效序列化、落盘和跨网络传输。
6. Context：业务代码与 Hadoop 运行时之间的桥梁。

一句话总结：写 MapReduce 程序时，你不是在手工编写整个分布式系统，而是在向 Hadoop 声明两件事：“每条数据怎样局部变换”和“同一个 Key 的数据怎样聚合”。其余复杂的分布式执行机制由框架负责。

下一讲预告

下一讲建议继续学习“MapReduce 的高级数据组织能力”：自定义 Partitioner、多个 Reducer、全局排序、Secondary Sort，以及数据倾斜的代码级解决方法。届时你会看到：MapReduce 真正高级的地方，不是把 reduce() 写得多复杂，而是控制“数据应该去哪里、以什么顺序到达”。